13: API security and RBAC

Kubernetes is API-centric, and the API is served through the *API server*. In this chapter, you'll follow a typical API request as it passes through various security-related checks.

The chapter is divided as follows.

- API security big picture
- Authentication
- Authorization (RBAC)
- Admission control

See chapter 14 for a detailed look at the API.

API security big picture

All of the following make CRUD-style requests to the API server (create, read, update, delete).

- Operators and developers using `kubectl`
- Pods
- Kubelets
- Control plane services
- More…

Figure 13.1 shows the flow of a typical API request passing through all the standard checks. The flow is the same no matter where the request originates.

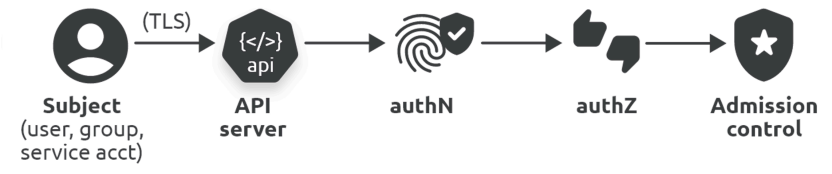

Figure 13.1

Consider a quick example where a user called "grant-ward" is trying to create a Deployment called "hive" in the "terran" Namespace.

User **grant-ward** issues a `kubectl` command to create the Deployment. This generates a request to the API server with the user's credentials embedded. Thanks to the magic of TLS, the connection between the client and the API server is secure. The *authentication* module determines whether it's **grant-ward** or an imposter. After that, the

authorization module (RBAC) determines whether **grant-ward** is allowed to create Deployments in the `terran` Namespace. If the request passes authentication and authorization, *admission control* checks and applies policies, and the request is finally accepted and executed.

It's a lot like flying on a plane. You travel to the airport and *authenticate* yourself with a photo ID, usually your passport. You then present a ticket *authorizing* you to board the plane and occupy a particular seat. If you pass authentication and are authorized to board, admission controls may then check and apply airline policies such as not taking hot food onboard, restricting your hand luggage, and prohibiting alcohol in the cabin. After all of that, you're finally allowed to board the plane and take your seat.

You're about to see things in more detail, but everything you'll learn assumes you're hitting the API on its *secured port.* Some Kubernetes clusters also expose the API on a local *unsecured port* on every control plane node. This bypasses TLS, authentication, and authorization, but it doesn't bypass admission control. Even though the unsecured port is only accessible if you can log on to a control plane node, you should probably disable it on most clusters.

Let's take a closer look at authentication.

Authentication

Authentication is about proving your identity. You might see or hear it shortened to *authN*, pronounced "auth en".

At the heart of authentication are credentials. **All requests to the API server have to include credentials**, and the authentication layer is responsible for verifying them. If verification fails, the API server returns an HTTP 401 and the request is denied. If it passes, it moves on to authorization.

The authentication layer in Kubernetes is pluggable, and popular modules include *client certs, webhooks,* and integration with external identity management systems such as *Active Directory (AD)* and cloud-based *Identity Access Management (IAM).* In fact, it's impossible to create user accounts in Kubernetes as it does **not** have its own built-in identity database. Instead, Kubernetes forces you to use an external system. This is great, as Kubernetes doesn't install yet another identity management silo.

Out-of-the-box, most Kubernetes clusters support *client certificates*, but in the real-world you'll want to integrate with your chosen cloud or corporate identity management system. Many of the hosted Kubernetes services make it easy to integrate with their native identity management systems.

Checking your current authentication setup

Cluster details and credentials are stored in a `kubeconfig` file. Tools like `kubectl` read this file to know which cluster to send commands to, as well as which credentials to use. It's usually stored in the following location.

- Windows: C:\Users<user>.kube\config
- Linux/Mac: /home/<user>/.kube/config

Many Kubernetes installations can automatically merge cluster endpoint details and credentials into your existing `kubeconfig`. For example, every GKE cluster provides a `gcloud` command that will merge the necessary cluster details and credentials to your local `kubeconfig` config file. The following is an example, don't try and run it.

```
$ gcloud container clusters get-credentials tkb --zone europe-west1-c --project thek8sbook
```

Here's what a `kubeconfig` file looks like. As you can see, it defines a *cluster* and a *user*, combines them into a *context*, and sets the default context for all `kubectl` commands. It's been snipped to fit the page.

```
apiVersion: v1
kind: Config
clusters:
- cluster:
  name: prod-shield
    server: https://<url-or-ip-address-of-api-server>:443
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0…LS0tCg==
users:
- name: njfury
  user:
    as-user-extra: {}
    token: eyJhbGciOiJSUzI1NiIsImtpZCI6IlZwMzl…SZY3uUQ
contexts:
- context:
  name: shield-admin
    cluster: prod-shield
    namespace: default
    user: njfury
current-context: shield-admin
```

You can see it's divided into `4` top-level sections.

The `clusters` section defines one or more Kubernetes clusters. Each one has a friendly name, an API server endpoint, and the public key of its certificate authority (CA). The cluster in the example is exposing the secure API endpoint on port `443` (HTTPS), but it's also common to see it exposed on `6443`.

The `users` section defines one or more users. Each user requires a name and token. The token is often an X.509 certificate that is the user's ID. If it is, it has to be signed by the cluster's CA or a CA trusted by the cluster.

The `contexts` section combines users and clusters, and the `current-context` is the cluster and user `kubectl` will use for all commands.

Assuming the previous `kubeconfig`, all `kubectl` commands will go to the "prod-shield" cluster and authenticate as the "njfury" user. The authentication module is responsible for determining if the user really is `njfury`, and if using client certificates, it'll determine if the certificate is signed by a trusted CA.

If your cluster integrates with an external IAM system, it'll hand-off authentication to that system.

Assuming authentication is successful, the request progresses to the authorization phase.

Authorization (RBAC)

Authorization happens immediately after successful authentication, and you'll sometimes see it shortened to *authZ* (pronounced "auth zee").

Kubernetes authorization is pluggable and you can run multiple authZ modules on a single cluster. As soon as any of the modules authorizes a request, it moves on to *admissions control*.

This section will cover the following.

- RBAC big picture
- Users and permissions
- Cluster-level users and permissions
- Pre-configured users and permissions

RBAC big picture

The most common authorization module is RBAC (Role-Based Access Control). At the highest level, it's about three things.

1. Users
2. Actions
3. Resources

Which *users* can perform which *actions* against which *resources*.

The following table shows a few examples.

User (subject)	Action	Resource
Bao	create	Pods
Kalila	list	Deployments
Josh	delete	ServiceAccounts

RBAC is enabled on most Kubernetes clusters and has been stable/GA since Kubernetes 1.8. It's a least-privilege deny-by-default system. This means all actions are denied by default, and you enable specific actions by creating *allow rules*. In fact, Kubernetes doesn't support *deny rules*, it only supports *allow rules*. This might seem like a small thing, but makes Kubernetes RBAC much simpler to implement and troubleshoot.

Users and Permissions

Two concepts are vital to understanding Kubernetes RBAC.

- Roles
- RoleBindings

Roles define a set of permissions, and RoleBindings grant those permissions to users.

The following resource manifest defines a Role object. It's called "read-deployments", and grants permission to `get, watch`, and `list` Deployment objects in the "shield" Namespace.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: shield
  name: read-deployments
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "watch", "list"]
```

On their own, Roles don't do anything. They need binding to a user.

The following RoleBinding grants the previous "read-deployments" Role to a user called "sky".

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-deployments
  namespace: shield
subjects:
- kind: User
  name: sky                   <<==== This is the authenticated user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: read-deployments      <<==== This is the Role to bind to the user
  apiGroup: rbac.authorization.k8s.io
```

If both of these are deployed to a cluster, an authenticated user called "sky" will be able to run commands such as `kubectl get deployments -n shield`.

It's important to understand that the username listed in the RoleBinding has to be a string, and has to match the username that was successfully authenticated.

Looking closer at rules

The previous Role object has three properties.

- apiGroups
- resources
- verbs

Together, they define which actions are allowed against which objects. `apiGroups` and `resources` define the object, and `verbs` define the actions. The example allows read access (`get`, `watch` and `list`) against Deployment objects.

The following table shows some `apiGroup` and `resources` combinations.

apiGroup	resource	Kubernetes API path
""	pods	/api/v1/namespaces/{namespace}/pods
""	secrets	/api/v1/namespaces/{namespace}/secrets
"storage.k8s.io"	storageclass	/apis/storage.k8s.io/v1/storageclasses
"apps"	deployments	/apis/apps/v1/namespaces/{namespace}/deployments

An empty set of double quotes ("") in the `apiGroups` field indicates the `core` API group. All other API sub-groups need specifying as a string enclosed in double quotes.

Kubernetes uses a standard set of ***verbs*** to describe the actions a subject can perform on a resource. Verb names are self-explanatory and case-sensitive. The following table lists them and demonstrates the REST-based nature of the API by showing how they map to HTTP methods. It also lists some common HTTP response codes.

HTTP method	Kubernetes verbs	Common responses
POST	create	201 created, 403 Access Denied
GET	get, list and watch	200 OK, 403 Access Denied
PUT	update	200 OK, 403 Access Denied
PATCH	patch	200 OK, 403 Access Denied
DELETE	delete	200 OK, 403 Access Denied

The **Kubernetes verbs** column lists the verbs you use in the rules section of a Role object.

Running the following command shows all API resources supported on your cluster. It also shows API group and supported verbs, and is a great resource for helping build rule definitions. The example has been trimmed to fit the page

```
$ kubectl api-resources --sort-by name -o wide
NAME          APIGROUP           KIND        VERBS
deployments   apps               Deployment  [create delete ... get list patch update watch]
ingresses     networking.k8s.io  Ingress     [create delete ... get list patch update watch]
pods                             Pod         [create delete ... get list patch update watch]
secrets                          Secret      [create delete ... get list patch update watch]
services                         Service     [create delete get list patch update watch]
```

If you compare the output columns with the `rules` block of the previous Role object, you see how things map.

```
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "watch", "list"]
```

You can use the asterisk (*) to refer to all API groups, all resources, and all verbs. For example, the following rule block grants all actions on all resources in every API group (basically cluster admin). It's just for demonstration purposes and you probably shouldn't create a rule like this.

```
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
```

Cluster-level users and permissions

So far, you've seen Roles and RoleBindings. However, Kubernetes actually has 4 RBAC objects.

- Roles
- ClusterRoles
- RoleBindings
- ClusterRoleBindings

Roles and *RoleBindings* are namespaced objects. This means they can only be applied to a single Namespace. *ClusterRoles* and *ClusterRoleBindings* are cluster-wide objects and apply to all Namespaces. All 4 are defined in the same API sub-group and their YAML structures are almost identical.

A powerful pattern is to define Roles at the cluster level (ClusterRoles) and bind them to specific Namespaces via RoleBindings. This lets you define common roles once, and re-use them across multiple Namespaces. For example, the following YAML defines the same "read-deployments" role, but this time at the cluster level. This can be re-used in selected Namespaces via RoleBindings.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: read-deployments
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "watch", "list"]
```

Look closely at the previous YAML. The only differences with the earlier one are that this one has its `kind` set to ClusterRole instead of Role, and it doesn't have a `metadata.namespace` property.

Figure 13.2 shows a role defined at the cluster level being applied to two Namespaces via two RoleBindings. It can easily be applied to the other Namespaces via two more RoleBindings. The role was defined once and re-used across multiple Namespaces.

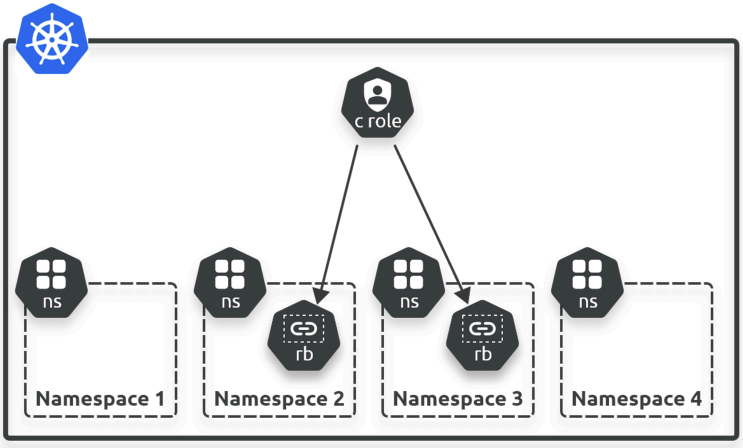

Figure 13.2

Pre-created users and permissions

To help with initial configuration and getting started, most clusters have a set of pre-created roles and bindings granting permissions to an all-powerful user. Many will also configure `kubectl` to operate under the context of that user.

The following example walks you through the pre-created user, roles and bindings on a Docker Desktop cluster. The names of Pods and RBAC objects will be different on other clusters, but the principles will be the same and it gives you an idea of how things are made to work.

Docker Desktop runs the API server in Pod in the `kube-system` Namespace. It has an `--authorization` flag that tells Kubernetes which authorization modules to use. The following command shows the node and RBAC modules are both enabled.

```
$ kubectl describe pod/kube-apiserver-docker-desktop \
  --namespace kube-system | grep authorization

      --authorization-mode=Node,RBAC
```

You won't be able to interrogate the API server like this on a hosted Kubernetes cluster. This is because critical control plane features like this are hidden from you.

Docker Desktop also updates your `kubeconfig` file with a user called `docker-desktop` and any necessary credentials.

```
$ kubectl config view
<Snip>
users:
- name: docker-desktop
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
<Snip>
```

RBAC is enabled, and a user and `kubeconfig` is created. Now let's look at the ClusterRole and ClusterRoleBinding objects preconfigured to grant permissions to the `docker-desktop` user.

The following command shows a ClusterRoleBinding called "docker-for-desktop-binding" that's bound to the "cluster-admin" ClusterRole.

```
$ kubectl get clusterrolebindings | grep docker
NAME                             ROLE
docker-for-desktop-binding       ClusterRole/cluster-admin
```

The following command `describes` the ClusterRoleBinding. As you can see, it's granting all members of the `system-serviceaccounts` group access to the pre-created `cluster-admin` role. As a result, the `docker-desktop` user configured in your `kubeconfig` file gets access to the all-powerful `cluster-admin` Role.

```
$ kubectl describe clusterrolebindings docker-for-desktop-binding
Name:         docker-for-desktop-binding
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  cluster-admin     <<==== This is the role it's bound to
Subjects:
  Kind   Name                    Namespace
  ----   ----                    ---------
  Group  system:serviceaccounts  kube-system  <<==== Members of this group get the binding
```

As a result of this binding, all commands in a default out-of-the-box Docker Desktop cluster are executed with `cluster-admin` permissions. This might be OK for development environments (which is what Docker Desktop is intended for) but it's not appropriate for production.

The following command shows the powers the cluster-admin role has – all *verbs* on all *resources* in all Namespaces.

```
$ kubectl describe clusterrole cluster-admin
Name:         cluster-admin
Labels:       kubernetes.io/bootstrapping=rbac-defaults
Annotations:  rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  ---------  -----------------  --------------  -----
  *.*        []                 []              [*]
             [*]                []              [*]
```

Summarising authorization

Authorization ensures authenticated users are allowed to carry out the actions they're attempting. RBAC is a popular Kubernetes authorization module and implements least privilege access based on a deny-by-default model where all actions are assumed to be denied unless a rule exists that allows it. The model is similar to a whitelist firewall where everything is blocked and you open up access by creating *allow rules*.

Kubernetes RBAC uses Roles and ClusterRoles to create permissions, and it uses RoleBindings and ClusterRoleBindings to grant those permissions to users.

Once a request passes authentication and authorization, it moves on to admission control.

Admission control

Admission control runs immediately after successful authentication and authorization, and it's all about *policies*.

There are two types of admission controllers.

- Mutating
- Validating

The names are self-explanatory. *Mutating* controllers check for compliance and can modify requests, whereas *validating* controllers check for policy compliance but cannot modify requests. Mutating controllers always run first, and both types only apply to requests that will modify state. Requests to read state are not subjected to admission control.

Assume a quick example where all new and updated objects to your cluster must have the `env=prod` label. A mutating controller can check for the presence of the label and add it if it doesn't exist. On the flip side, a validating controller can only reject the request if it doesn't exist.

The following command on a Docker Desktop cluster shows the API server is configured to use the `NodeRestriction` admission controller.

```
$ kubectl describe pod/kube-apiserver-docker-desktop \
  --namespace kube-system | grep admission

--enable-admission-plugins=NodeRestriction
```

Most real-world clusters will have a lot more admission controllers enabled.

There are lots of admission controllers, but the `AlwaysPullImages` controller is a great example. It's a mutating controller that sets the `spec.containers.imagePullPolicy` of all new Pods to "Always". This means the images for all containers in all Pods will always be pulled from the registry. This accomplishes quite a few things, including the following.

- Preventing the use of locally cached images that could be malicious
- Preventing other Pods and processes using locally cached images
- Forcing the container runtime to present valid credentials to the registry to get the image

If any admission controller rejects a request, the request is immediately rejected without checking other admission controllers. However, if all admission controllers approve the request, it gets persisted to the cluster store and instantiated on the cluster.

As previously mentioned, there are lots of admission controllers, and they're becoming more and more important in real-world production clusters.

Chapter summary

In this chapter, you learned that all requests to the API server include credentials and pass through authentication, authorization, and admission control. The connection between the client and the API server is also secured with TLS.

The authentication layer is responsible for validating the identity of requests. Client certificates are commonly used, and integration with AD and other IAM services is recommended for production clusters. Kubernetes does not have its own identity database, meaning it doesn't store or manage user accounts.

The authorization layer checks whether the authenticated user is authorized to carry out the action in the request. This layer is also pluggable and the most common module is RBAC. RBAC comprises 4 objects that define permissions and assign them to users.

Admission control kicks in after authorization and is responsible for enforcing policies. Validating admission controllers reject requests if they don't conform to policy, whereas mutating admission controllers can modify requests to enforce policies.